

Aufgabe 1: Ankreuzfragen (24 Punkte)

1) Einfachauswahlfragen (16 Punkte)

Bei den Einfachauswahlfragen in dieser Aufgabe ist jeweils nur **eine** richtige Antwort eindeutig anzukreuzen. Auf die richtige Antwort gibt es die angegebene Punktzahl.

Wollen Sie eine Antwort korrigieren, streichen Sie bitte die falsche Antwort mit drei waagrechten Strichen durch (~~☒~~) und kreuzen die richtige an.

Lesen Sie die Frage genau, bevor Sie antworten.

a) Gegeben seien die folgenden Präprozessor-Makros:

```
#define ADD(a, b) a + b
```

```
#define MUL(a, b) a * b
```

Was ist das Ergebnis des folgenden Ausdrucks? `2 * MUL ( ADD(6,4) , 3)`

2 Punkte

- ☐ 60
☐ 48
☐ 36
☐ 24

b) Gegeben sei folgende C-Funktion:

```
void func(void) {  
    static int a = 42;  
    // [...]  
}
```

2 Punkte

Welche Aussage zu Lebensdauer und Sichtbarkeit der Variablen `a` ist korrekt?

- ☐ Lebensdauer: Programmlaufzeit, Sichtbarkeit: global
☐ Lebensdauer: Funktionslaufzeit, Sichtbarkeit: global
☐ Lebensdauer: Funktionslaufzeit, Sichtbarkeit: funktionslokal
☐ Lebensdauer: Programmlaufzeit, Sichtbarkeit: funktionslokal

c) Was passiert, wenn Sie in einem C-Programm über einen ungültigen Zeiger versuchen auf Speicher zuzugreifen?

2 Punkte

- ☐ Die MMU erkennt die ungültige Adresse bei der Adressumsetzung und löst einen Trap aus.
☐ Beim Laden des Programms wird die ungültige Adresse erkannt und der Speicherzugriff durch einen Sprung auf eine Abbruchfunktion ersetzt. Diese Funktion beendet das Programm mit der Meldung „*Segmentation fault*“.
☐ Der Compiler erkennt die problematische Code-Stelle und generiert Code, der zur Laufzeit bei dem Zugriff einen entsprechenden Fehler auslöst.
☐ Der Arbeitsspeicher erkennt, dass es sich um eine ungültige Adresse handelt und leitet eine Ausnahmebehandlung ein.

d) Welche der folgenden Aussagen zum Thema Threads und Prozesse ist richtig?

2 Punkte

- ☐ Die Umschaltung von User-Level Threads (*feather-weight processes*) ist eine privilegierte Operation und muss deshalb im Systemkern erfolgen.
☐ Threads (*light-weight processes*) teilen sich den kompletten Adressraum und verwenden daher den selben Stack.
☐ Die Erzeugung eines Threads (*light-weight process*) ist teurer als die Erzeugung eines User-Level Threads (*feather-weight process*).
☐ Zu jedem Thread (*light-weight process*) gehört ein eigener, isolierter Adressraum.

e) Ein Prozess wird in den Zustand bereit (*ready*) überführt. Welche Aussage passt zu diesem Vorgang?

2 Punkte

- ☐ Der Prozess hat auf eine Tastatureingabe gewartet und diese steht nun zur Verfügung.
☐ Der Prozess hat einen Seitenfehler für eine Seite, die aber noch im Hauptspeicher vorhanden ist.
☐ Der Prozess wartet auf Daten von der Festplatte.
☐ Ein anderer Prozess blockiert sich an einem Semaphor.

f) Welche Aussage zum Thema Adressräume ist richtig?

2 Punkte

- ☐ Die Größe eines virtuellen Adressraums darf die Größe des vorhandenen Hauptspeichers nicht überschreiten.
☐ In einem virtuellen Adressraum sind alle Adressen gültig, es müssen jedoch nicht zu jedem Zeitpunkt alle Daten im Hauptspeicher vorliegen.
☐ Ein Programm kann ohne weiteres auf den Inhalt der Adressräume anderer Prozesse zugreifen.
☐ Die Abbildung von virtuellen Adressen auf physikalische Adressen erfolgt zur Laufzeit durch den Compiler.

g) Sie kennen den Translation Lookaside Buffer (TLB). Welche Aussage ist richtig?

2 Punkte

- ☐ Der TLB puffert die Ergebnisse der Abbildung von physikalischen auf virtuelle Adressen, sodass eine erneute Anfrage sofort beantwortet werden kann.
- ☐ Wird eine Speicherabbildung im TLB nicht gefunden, wird der auf den Speicher zugreifende Prozess mit einer Schutzraumverletzung (*Segmentation Fault*) abgebrochen.
- ☐ Der TLB ist ein sehr schneller Zwischenpuffer der MMU, der virtuelle in physikalische Adressen umsetzt.
- ☐ Der TLB verkürzt die Zugriffszeit auf den physikalischen Speicher, da ein Teil des Speichers in einem sehr schnellen Pufferspeicher vorgehalten wird.

h) Beim Einsatz von RAID-Systemen kann durch zusätzliche Festplatten Fehlertoleranz erzielt werden. Welche Aussage dazu ist richtig?

2 Punkte

- ☐ Bei RAID 1 liegen die Paritätsinformationen auf einer dezidierten Platte.
- ☐ Bei RAID 5 kann der gleichzeitige Ausfall mehrerer Platten kompensiert werden.
- ☐ Bei RAID 0 führt der Ausfall einer der beteiligten Platten zu Datenverlust.
- ☐ Bei RAID 4 Systemen werden Paritätsinformationen gleichmäßig über alle beteiligten Platten verteilt.

2) Mehrfachauswahlfragen (8 Punkte)

Bei den Mehrfachauswahlfragen in dieser Aufgabe sind jeweils m Aussagen angegeben, davon sind n ($0 \leq n \leq m$) Aussagen richtig. Kreuzen Sie alle richtigen Aussagen an.

Jede korrekte Antwort in einer Teilaufgabe gibt einen Punkt, jede falsche Antwort einen Minuspunkt. Eine Teilaufgabe wird minimal mit 0 Punkten gewertet, d. h. falsche Antworten wirken sich nicht auf andere Teilaufgaben aus.

Wollen Sie eine falsch angekreuzte Antwort korrigieren, streichen Sie bitte das Kreuz mit drei waagrechten Strichen durch (~~☒~~).

Lesen Sie die Frage genau, bevor Sie antworten.

a) Konzeptionell ist der Speicher eines UNIX-Prozesses in Text-, Daten- und Stack-Segment untergliedert. Welche der folgenden Aussagen treffen zu?

4 Punkte

- ☐ Lokale Variablen der Speicherklasse *static* werden bei jedem Betreten der zugehörigen Funktion neu initialisiert.
- ☐ Dynamisch mittels `malloc(3)` allozierte Zeichenketten werden in das Heap-Segment gelegt.
- ☐ Der Code von Funktionen wird zusammen mit den Variablen der Funktion im Stack-Segment abgelegt.
- ☐ Variablen der Speicherklasse *static* liegen im Daten-Segment.
- ☐ Bei einem `malloc(3)`-Aufruf wird das Stack-Segment dynamisch erweitert.
- ☐ Das Text-Segment enthält sowohl den Programmcode als auch konstante Zeichenketten.
- ☐ Zeigervariablen können auf Daten aus allen Segmenten verweisen.
- ☐ Auf Variablen im Daten-Segment kann nur lesend zugegriffen werden.

b) Welche der folgenden Aussagen zu prioritätsbasierten Scheduling-Verfahren sind richtig?

4 Punkte

- ☐ Prioritätsumkehr (*priority inversion*) meint, dass ein Prozess mit höherer Priorität durch einen Prozess mit niedrigerer Priorität blockiert wird, wenn dieser eine Ressource exklusiv belegt.
- ☐ Prioritätsumkehr (*priority inversion*) meint, dass ein Prozess mit niedriger Priorität abgebrochen wird (*umkehrt*), damit ein Prozess mit höherer Priorität laufen kann.
- ☐ Prioritätsumkehr (*priority inversion*) kann nur mit dynamischen Prioritäten auftreten.
- ☐ Prioritätsumkehr (*priority inversion*) kann durch Prioritätsgrenzen (*priority ceiling protocols*) verhindert werden.
- ☐ Bei Multilevel Feedback Queues werden Prozesse, die zu lange laufen, abgebrochen (*anti-aging*), um sicherzustellen, dass andere Prozesse nicht verhungern (*starvation*).
- ☐ Kurze Prozesse werden bei Multilevel Feedback Queues generell bevorzugt.
- ☐ Bei Multilevel Feedback Queues können Prozesse, die schon lange auf Ausführung warten ggf. in Queues mit höherer Priorität zurückwechseln (*anti-aging*).
- ☐ Multilevel Feedback Queues können nur bei statischen Prioritäten eingesetzt werden.

Aufgabe 2: chatOS (45 Punkte)

Sie dürfen diese Seite zur besseren Übersicht bei der Programmierung heraustrennen!

Schreiben Sie ein POSIX-1.2008 und C11-konformes Programm chatOS, welches ein Chatinterface für Ihr Betriebssystem implementiert. Wir befinden uns noch am Anfang der Chat-Betriebssystementwicklung, daher kann chatOS bislang nur Fragen und Kommandos ausführen, die interaktiv per Standardeingabe eingegeben werden. Fragen werden per Funktionsaufruf durch eine künstliche Intelligenz ausgewertet und dann ausgegeben. Kommandos werden lokal ausgeführt.

Beispielhafter Aufruf von chatOS: hofmeier@tardis:~\$./chatOS

Beispielhafte Interaktion mit chatOS:

ChatOS: How can I help you?	ChatOS: How can I help you?
What is the answer to everything?	ls!
ChatOS: 42	k9.txt weeping_angels.exe

Das Programm soll folgendermaßen strukturiert sein:

– Funktion **int** main(**void**):

Es wird zunächst die Eingabeaufforderung (**char** *prompt) ausgegeben. Im Anschluss wird die Eingabe **zeichenweise** eingelesen, wobei es **keine** Begrenzung der Zeilenlänge gibt. Eine Eingabe wird mit einem Zeilenumbruch abgeschlossen. Wird **EOF** gelesen, wird das Programm beendet. Leere Zeilen sollen kommentarlos ignoriert werden. Nachdem eine Zeile eingelesen wurde, wird die Art der Eingabe bestimmt:

Bei einer **Frage** endet die Zeile mit einem Fragezeichen ('?'). Zur Auswertung der Frage wird die eingelesene Zeile an die Funktion handle_question() (siehe unten) übergeben.

Ein **Kommando** hat ein Ausrufezeichen ('!') am Zeilenende. Die eingelesene Zeile wird zur Ausführung des Kommandos an die Funktion handle_command() (siehe unten) übergeben.

Wird die Eingabe weder als Frage noch als Kommando erkannt, ist sie ungültig. Bei einer ungültigen Eingabe wird eine Fehlermeldung ausgegeben und die Ausführung mit einer neuen Eingabe fortgesetzt.

– Funktion **void** handle_command(**char*** line):

Die Funktion erhält als Parameter die eingelesene Zeile (line). Die Zeile wird zunächst in Kommandonamen und Argumente zerlegt. Trennzeichen sind die mittels dem globalen **char** *delim definierten Zeichen. Wird kein Kommando gefunden, kehrt die Funktion ohne Fehlermeldung zurück. Im Anschluss wird ein neuer Kindprozess erzeugt und das Kommando mithilfe einer Funktion der exec()-Familie ausgeführt. Der Elternprozess wartet bis der erzeugte Kindprozess terminiert und kehrt anschließend zurück.

– Funktion **void** handle_question(**char** *line):

Die Funktion sendet die eingelesene Zeile (line) an die künstliche Intelligenz zur Auswertung. Hierfür wird die Funktion answer_question(**char** *string) genutzt, die über die Schnittstelle ai.h zur Verfügung steht. answer_question() gibt einen Zeiger auf den Antwortstring zurück. Die Antwort wird auf der Standardausgabe ausgegeben.

Hinweise:

- Es ist **keine** Fehlerbehandlung/fflush() für Ausgaben nötig. Achten Sie ansonsten auf korrekte und vollständige Fehlerbehandlung.
- Eine Beschreibung der ai.h Schnittstelle und deren Funktion answer_question() finden Sie in den angehängten Manpages.

Hinweis: mit der Nutzung der gegebenen Trennzeichen (char *delim) in handle_command() wird nicht nur das '!' am Ende Zeile entfernt sondern u.U. auch '!' in den Kommandoparametern. Wir

nehmen vereinfachend an, dass bei gültigen Kommandoaufrufen das '!' nur am Zeilenende vorkommt.

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#include "ai.h"

static char *delim = "\t\n!";
static char *prompt = "\nChatOS: How can I help you?\n";

static void die(const char msg[]) {
    perror(msg);
    exit(EXIT_FAILURE);
}

static void err(const char msg[]) {
    fprintf(stderr, "%s", msg);
    exit(EXIT_FAILURE);
}

// Makros, Funktionsdeklarationen, globale Variablen

// Funktion main

// Hauptschleife

// Prompt
```

```
// Befehl von stdin einlesen.
```

```
// Art der Eingabe bestimmen
```

10

```
// Aufräumen
```

7

```
// Ende Funktion main
```

M:

```
// Funktion handle_command
```

```
// Befehlszeile parsen
```

10

10

10

```
// Kindprozess starten
```

10

```
// Auf Kindprozess warten
```

```
// Ende Funktion handle_command
```

C:

```
// Funktion handle_question
```

7

```
// Frage einreichen
```

--	--

```
// Antwort ausgeben
```

7

```
// Ende Funktion handle_question
```

Q:

Vervollständigen Sie das folgende Makefile. Es soll ein Target `chat05` unterstützt werden, welches das Programm `chat05` baut. Greifen Sie dabei stets auf Zwischenprodukte (z.B. `chat05.o`) zurück. Gehen Sie davon aus, dass das vorgegebene Modul `ai.o` stets vorliegt und daher nicht erzeugt werden muss.

Nutzen Sie dabei die Variablen `CC` und `CFLAGS` konventionskonform. Achten Sie darauf, dass das Makefile ohne eingebaute Variablen und Regeln (Aufruf von `make -Rr`) funktioniert!

.PHONY: all clean

CC=gcc

```
CFLAGS=-std=c11 -pedantic -Wall -D_XOPEN_SOURCE=700
```

```
all: chat05
```

clean:

```
rm -f chatOS chatOS.o
```

10

11

Mk:

Aufgabe 3: Synchronisation (9 Punkte)

1) Skizzieren Sie in Programmiersprachen-ähnlicher Form, wie das folgende Szenario mithilfe eines Semaphors korrekt synchronisiert werden kann: Sieben Threads durchsuchen mithilfe der Funktion `searchQuery()` parallel das Dateisystem nach Suchtreffern. Die Anzahl der Treffer werden auf den Wert einer globalen Variable `hits` aufaddiert. Zu jedem Zeitpunkt müssen so viele Threads wie möglich die Funktion `searchQuery()` parallel ausführen können. Ihnen stehen dabei folgende Semaphor-Funktionen zur Verfügung: (5 Punkte)

- `SEM * semCreate(int);`
- `void P(SEM *);`
- `void V(SEM *);`

Beachten Sie, dass für eine korrekte Lösung nicht unbedingt in allen Zeilen eine Operation vorgenommen werden muss. **Kennzeichnen Sie durch `/'`, wenn Ihre Implementierung in einer freien Zeile keine Operation benötigt.**

Hauptthread:

```
static int hits;
static SEM *s;
int main(void){

-----

    for(int i = 0; i < 6; ++i) {
        startWorkerThread(threadFunc);
    }

-----

    while(1) {

-----

        int n = searchQuery();

-----

        hits += n;

-----

    }

}
```

Arbeiterthread:

```
void threadFunc(void) {

-----

    while(1) {

-----

        int n = searchQuery();

-----

        hits += n;

-----

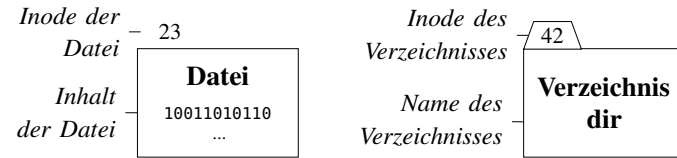
    }

}
```

2) Was versteht man unter einseitiger Synchronisation (*unilateral synchronisation*)? (2 Punkte)

3) Was versteht man unter einer Verklemmung (*Deadlock*)? (2 Punkte)

Ein Dateisystem ermöglicht das strukturierte Ablegen von Daten. Nachfolgend ist ein Verzeichnisbaum abgebildet. Verzeichnisse und Dateien sind mit einem beschrifteten Rechteck gekennzeichnet und wie folgt aufgebaut: *Inode der* *Inode des*



```

graph TD
    Root["Wurzelverzeichnis (/)  
2"] --> Bin["Verzeichnis bin  
63"]
    Root --> Etc["Verzeichnis etc  
14"]
    Root --> Home["Verzeichnis home  
52"]
    Bin -- ls --> BinDatei["Datei  
10011010110  
...  
5"]
    Etc -- which --> EtcDatei["Datei  
10011010110  
...  
67"]
    Home -- wsort --> HomeDatei["Datei  
10011010110  
...  
30"]
    Bin -- alt-which --> EtcDatei
  
```

	Wurzelverzeichnis	bin	etc	home
Inode des Verzeichnisses			14	
			14	.
			2	..
Inode und Name des Eintrags			67	which

Eine Anpassung der Verwaltungsinformationen aus Teilaufgabe 1) ist **nicht** erforderlich.

3) Journaling Dateisysteme (*Journaled File Systems*) nutzen zur Ausführung von Dateisystemänderungen Transaktionen und ein Log, um die Fehlertoleranz und Ausfallsicherheit zu erhöhen. Beschreiben Sie **stichpunktartig** die dafür nötigen Schritte bei der Protokollierung (*Protocol*) der Dateisystemoperationen und wie das Betriebssystem im Falle der Wiederherstellung (*Recovery*) beim Boot vorgeht. (4 Punkte)

[illegible]